TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

Professur
Betriebssysteme

Übung 3 – Typenbegriff, Skalare und Traits

Algorithmen und Programmierung

Martin Richter, Maik Benndorf, Bastian Felix Bachmann, Jens Pönisch

Aufgabe 1 (Vielfaches)

Erstellen Sie eine rekursive Funktion `is_multiple(larger: f32, smaller: f32)-> bool` , welche feststellt ob `larger` ein Vielfaches von `smaller` ist. Worauf muss man beim Vergleich von Gleitkommazahlen achten?

Aufgabe 2 (Typen)

Implementieren Sie eine Funktion `print_type`, welche für einen beliebigen Wert den Typ des Wertes auf der Kommandozeile ausgibt (es kommen nur die bisher kennengelernten (primitiven) Typen in Frage). Welches Konzept aus der Vorlesung benötigen Sie dafür?

Aufgabe 3

Implementieren Sie eine Funktion `is_large` welche für einen Wert eines beliebigen (Zahlen-) Typs zurückgibt, ob der Wert verglichen zum Wertebereich des Typs groß oder klein ist (`true` bedeutet groß und `false` klein). Groß ist er, wenn er in der oberen Hälfte des darstellbaren Zahlenraumes liegt (inklusive der Mitte).

Aufgabe 4

Implementieren Sie den folgenden Trait `Power` für `f64` und `u64`.

```
trait Power {
    fn pow(self, exp: u32) -> Self;
}
```

Die Methode `pow` soll dabei self^{exp} berechnen.

Aufgabe 5

Implementieren Sie den folgenden Trait `DigitCounter` für `f64` und `u64`.

```
trait DigitCounter {
    fn count_digits(self) -> usize;
}
```

Die Methode `count_digits` soll dabei die Anzahl der Stellen der entsprechenden Zahl berechnen. Bei Gleitkommazahlen ergibt sich diese aus der Summe der Anzahl der Vorkommastellen und der Anzahl der Nachkommastellen (wobei die Zahl auf 10000-stel gerundet werden soll). Falls der Nachkommateil 0 beträgt (z.B. 10.0), zählt das als 0 Nachkommastellen.

Aufgabe 6 (Verschlüsselung)

Gegeben sei folgender Trait `CryptoMessage`:

```
trait CryptoMessage {
    fn encrypt(self, key: u8) -> Self;
    fn decrypt(self, key: u8) -> Self;
    fn print_bytes(self);
}
```

Er steht für die Fähigkeit eines Typen Byte-weise verschlüsselt zu werden mithilfe der Caesar-Chiffre[1]. Die Methoden `encrypt` und `decrypt` werden jeweils für die Ver- und Entschlüsselung genutzt. Die Methode `print_bytes` wird für die Byte-weise Ausgabe der Nachricht genutzt. Dabei soll der Schlüssel `key` auf die Nachricht (für die der Trait implementiert ist) byte-weise modulo-aufaddiert werden. Das heißt für die Verschlüsselung enc einer Nachricht M, bestehend aus mehreren Bytes b, und einen Schlüssel k:

$$\forall b \in M : \mathrm{enc}(b, k) = (b + k) \bmod 256$$

Die Entschlüsselung findet analog für eine verschlüsselte Nachricht M' statt:

$$\forall b \in M' : \mathrm{dec}(b, k) = (b - k) \bmod 256$$

[1] `https://de.wikipedia.org/wiki/Caesar-VerschlÃsselung`